

LAPORAN PRAKTIKUM PEMROGRAMAN WEB
Modul 8: Pengantar Framework & MVC Architecture + CodeIgniter 3



**Universitas
Telkom**

Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S

NIM: 707012530010

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

A. Tujuan

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Memahami konsep dan fungsi framework dalam pengembangan aplikasi web.
2. Memahami arsitektur MVC (Model-View-Controller).
3. Menjelaskan arsitektur dan konsep dasar CodeIgniter 3.
4. Menginstal dan mengonfigurasi CodeIgniter 3.
5. Membuat routing dan controller pada CI3.
6. Mengimplementasikan view dan template system.
7. Menghubungkan model dengan database.
8. mengintegrasikan komponen MVC menjadi aplikasi utuh.
9. Mengimplementasikan form handling dan validation.
10. Mengelola session, autentikasi, dan role management.
11. Menerapkan praktik keamanan dasar pada aplikasi CI3.

B. Alat dan Bahan

Untuk melaksanakan praktikum ini, diperlukan:

1. Laptop/PC dengan XAMPP terinstal.
2. PHP versi minimal 7.x.
3. Database MySQL/MariaDB.
4. Code Editor (VS Code/Sublime/PHPStorm).
5. Browser (Chrome/Firefox).
6. CodeIgniter 3 (download dari <https://codeigniter.com>).

C. Langkah-Langkah Praktikum

1. Langkah pertama yang dilakukan adalah menyiapkan lingkungan kerja dengan menjalankan Apache dan MySQL pada XAMPP.
2. Selanjutnya project CodeIgniter 3 diletakkan pada direktori htdocs dengan nama folder ci3_project.
3. Setelah itu dibuat database dengan nama webpro melalui phpMyAdmin.
4. Database kemudian dihubungkan dengan aplikasi melalui pengaturan pada file application/config/database.php.
5. Untuk menyiapkan struktur data yang dibutuhkan, dilakukan import file webpro_ci3.sql ke dalam database webpro.
6. File SQL tersebut digunakan untuk membuat tabel ci3_users sebagai tabel pengguna dan ci3_participants sebagai tabel data peserta.
7. Tahap berikutnya adalah mengatur routing aplikasi pada file application/config/routes.php agar URL dapat mengarah ke halaman login, register, dashboard, peserta, dan logout.
8. Kemudian dilakukan pengaturan autoload pada file application/config/autoload.php untuk memuat library dan helper yang dibutuhkan, seperti database, session, form_validation, upload, url, dan form.
9. Setelah konfigurasi selesai, dibuat controller Auth.php untuk menangani proses

autentikasi pengguna, yaitu login, register, dan logout.

10. Selanjutnya dibuat User_model.php yang berfungsi untuk mengelola data pengguna pada database.
11. Setelah fitur autentikasi selesai, dibuat controller Dashboard.php untuk menampilkan halaman utama aplikasi yang hanya dapat diakses setelah proses login berhasil.
12. Pada halaman dashboard ditampilkan ringkasan data peserta sebagai informasi utama setelah pengguna masuk ke sistem.
13. Selanjutnya dibuat Participant_model.php untuk mengelola seluruh proses pengambilan, penambahan, pengubahan, dan penghapusan data peserta.
14. Controller Peserta.php kemudian dibuat untuk mengatur proses CRUD data peserta sekaligus menangani upload file foto peserta.
15. Pada proses upload file, diterapkan validasi tipe file dan batas ukuran file agar file yang diunggah sesuai ketentuan sistem.
16. Form data peserta dirancang menggunakan satu tombol Simpan yang berfungsi untuk menyimpan data peserta sekaligus mengunggah file foto dalam satu kali submit, sesuai instruksi modul praktikum.
17. Setelah proses backend selesai, dibuat tampilan login pada file application/views/auth/login.php.
18. Selain itu, dibuat juga tampilan register pada file application/views/auth/register.php.
19. Halaman dashboard dibuat pada file application/views/dashboard/index.php.
20. Halaman kelola peserta dibuat pada file application/views/participants/index.php yang berisi tabel data peserta, tombol tambah, edit, hapus, dan form upload foto.
21. Untuk menampilkan pesan sistem, dibuat komponen notifikasi pada file application/views/partials/alerts.php.
22. Agar tampilan aplikasi lebih menarik, ditambahkan file CSS pada folder assets dan file JavaScript untuk mendukung interaksi form modal.
23. Setelah seluruh komponen selesai dibuat, aplikasi dijalankan melalui browser menggunakan alamat http://localhost/ci3_project/.
24. Tahap akhir praktikum adalah melakukan pengujian seluruh fitur, mulai dari register, login, akses dashboard, tambah data peserta, edit data, hapus data, dan upload foto.
25. Hasil dari setiap pengujian kemudian didokumentasikan dalam bentuk screenshot untuk dilampirkan pada laporan praktikum.

D. Hasil dan Dokumentasi Program

Bagian Konfigurasi

config.php

```
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

/*
|-----|
| Base Site URL
|-----|
|
| URL to your CodeIgniter root. Typically this will be your base URL,
| WITH a trailing slash:
|
|   http://example.com/
|
| If this is not set then CodeIgniter will try guess the protocol, domain
| and path to your installation. However, you should always configure this
| explicitly and never rely on auto-guessing, especially in production
| environments.
|
|*/
$config['base_url'] = ((isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] == "on")
? "https" : "http");
$config['base_url'] .= "://".$_SERVER['HTTP_HOST'];
$config['base_url'] .=
str_replace(basename($_SERVER['SCRIPT_NAME']),"",$_SERVER['SCRIPT_NAME']);

date_default_timezone_set('Asia/Jakarta');

/*
|-----|
| Index File
|-----|
|
| Typically this will be your index.php file, unless you've renamed it to
| something else. If you are using mod_rewrite to remove the page set this
| variable so that it is blank.
|
|*/
$config['index_page'] = '';

/*
|-----|
| URI PROTOCOL
|-----|
|
| This item determines which server global should be used to retrieve the
| URI string. The default setting of 'REQUEST_URI' works for most servers.
| If your links do not seem to work, try one of the other delicious flavors:
|
| 'REQUEST_URI'    Uses $_SERVER['REQUEST_URI']
| 'QUERY_STRING'   Uses $_SERVER['QUERY_STRING']
| 'PATH_INFO'      Uses $_SERVER['PATH_INFO']
|
| WARNING: If you set this to 'PATH_INFO', URIs will always be URL-decoded!
```

```
*/
$config['uri_protocol'] = 'REQUEST_URI';

/*
|-----
| URL suffix
|-----
|
| This option allows you to add a suffix to all URLs generated by CodeIgniter.
| For more information please see the user guide:
|
| http://codeigniter.com/user\_guide/general/urls.html
*/
$config['url_suffix'] = '';

/*
|-----
| Default Language
|-----
|
| This determines which set of language files should be used. Make sure
| there is an available translation if you intend to use something other
| than english.
*/
$config['language'] = 'english';

/*
|-----
| Default Character Set
|-----
|
| This determines which character set is used by default in various methods
| that require a character set to be provided.
|
| See http://php.net/htmlspecialchars for a list of supported charsets.
*/
$config['charset'] = 'UTF-8';

/*
|-----
| Enable/Disable System Hooks
|-----
|
| If you would like to use the 'hooks' feature you must enable it by
| setting this variable to TRUE (boolean). See the user guide for details.
*/
$config['enable_hooks'] = FALSE;

/*
|-----
|
```

```

| Class Extension Prefix
|-----
|
| This item allows you to set the filename/classname prefix when extending
| native libraries. For more information please see the user guide:
|
| http://codeigniter.com/user\_guide/general/core\_classes.html
| http://codeigniter.com/user\_guide/general/creating\_libraries.html
|
| */
$config['subclass_prefix'] = 'MY_';
/*
|-----
| Composer auto-loading
|-----
|
| Enabling this setting will tell CodeIgniter to look for a Composer
| package auto-loader script in application/vendor/autoload.php.
|
| $config['composer_autoload'] = TRUE;
|
| Or if you have your vendor/ directory located somewhere else, you
| can opt to set a specific path as well:
|
| $config['composer_autoload'] = '/path/to/vendor/autoload.php';
|
| For more information about Composer, please visit http://getcomposer.org/
|
| Note: This will NOT disable or override the CodeIgniter-specific
| autoloading (application/config/autoload.php)
| */
$config['composer_autoload'] = FALSE;
/*
|-----
| Allowed URL Characters
|-----
|
| This lets you specify which characters are permitted within your URLs.
| When someone tries to submit a URL with disallowed characters they will
| get a warning message.
|
| As a security measure you are STRONGLY encouraged to restrict URLs to
| as few characters as possible. By default only these are allowed: a-z 0-
9~%.:_-
|
| Leave blank to allow all characters -- but only if you are insane.
|
| The configured value is actually a regular expression character group
| and it will be executed as: ! preg_match('/^[<permitted_uri_chars>]+$/i
|
| DO NOT CHANGE THIS UNLESS YOU FULLY UNDERSTAND THE REPERCUSSIONS!!

```

```

|
| */
| $config['permitted_uri_chars'] = 'a-z 0-9~%.:_-';
|
| /*
| -----
| Enable Query Strings
| -----
|
| By default CodeIgniter uses search-engine friendly segment based URLs:
| example.com/who/what/where/
|
| By default CodeIgniter enables access to the $_GET array. If for some
| reason you would like to disable it, set 'allow_get_array' to FALSE.
|
| You can optionally enable standard query string based URLs:
| example.com?who=me&what=something&where=here
|
| Options are: TRUE or FALSE (boolean)
|
| The other items let you set the query string 'words' that will
| invoke your controllers and its functions:
| example.com/index.php?c=controller&m=function
|
| Please note that some of the helpers won't work as expected when
| this feature is enabled, since CodeIgniter is designed primarily to
| use segment based URLs.
|
| */
| $config['allow_get_array'] = TRUE;
| $config['enable_query_strings'] = FALSE;
| $config['controller_trigger'] = 'c';
| $config['function_trigger'] = 'm';
| $config['directory_trigger'] = 'd';
|
| /*
| -----
| Error Logging Threshold
| -----
|
| You can enable error logging by setting a threshold over zero. The
| threshold determines what gets logged. Threshold options are:
|
| 0 = Disables logging, Error logging TURNED OFF
| 1 = Error Messages (including PHP errors)
| 2 = Debug Messages
| 3 = Informational Messages
| 4 = All Messages
|
| You can also pass an array with threshold levels to show individual error
| types
|
| array(2) = Debug Messages, without Error Messages

```

```
|
| For a live site you'll usually only enable Errors (1) to be logged otherwise
| your log files will fill up very fast.
|
| */
$config['log_threshold'] = 0;
|
| /*
| -----
| Error Logging Directory Path
| -----
|
| Leave this BLANK unless you would like to set something other than the
| default
| application/logs/ directory. Use a full server path with trailing slash.
|
| */
$config['log_path'] = '';
|
| /*
| -----
| Log File Extension
| -----
|
| The default filename extension for log files. The default 'php' allows for
| protecting the log files via basic scripting, when they are to be stored
| under a publicly accessible directory.
|
| Note: Leaving it blank will default to 'php'.
|
| */
$config['log_file_extension'] = '';
|
| /*
| -----
| Log File Permissions
| -----
|
| The file system permissions to be applied on newly created log files.
|
| IMPORTANT: This MUST be an integer (no quotes) and you MUST use octal
| integer notation (i.e. 0700, 0644, etc.)
|
| */
$config['log_file_permissions'] = 0644;
|
| /*
| -----
| Date Format for Logs
| -----
|
| Each item that is logged has an associated date. You can use PHP date
| codes to set your own date formatting
|
```



```

*/
$config['log_date_format'] = 'Y-m-d H:i:s';

/*
|-----
| Error Views Directory Path
|-----
|
| Leave this BLANK unless you would like to set something other than the
default
| application/views/errors/ directory. Use a full server path with trailing
slash.
|
*/
$config['error_views_path'] = '';

/*
|-----
| Cache Directory Path
|-----
|
| Leave this BLANK unless you would like to set something other than the
default
| application/cache/ directory. Use a full server path with trailing slash.
|
*/
$config['cache_path'] = '';

/*
|-----
| Cache Include Query String
|-----
|
| Whether to take the URL query string into consideration when generating
| output cache files. Valid options are:
|
| FALSE      = Disabled
| TRUE       = Enabled, take all query parameters into account.
|              Please be aware that this may result in numerous cache
|              files generated for the same page over and over again.
| array('q') = Enabled, but only take into account the specified list
|              of query parameters.
|
*/
$config['cache_query_string'] = FALSE;

/*
|-----
| Encryption Key
|-----
|
| If you use the Encryption class, you must set an encryption key.
| See the user guide for more info.

```

```

| http://codeigniter.com/user_guide/libraries/encryption.html
|
| */
$config['encryption_key'] = 'webpro4904-ci3-incluskill';
/*
|-----
| Session Variables
|-----
|
| 'sess_driver'
|
|     The storage driver to use: files, database, redis, memcached
|
| 'sess_cookie_name'
|
|     The session cookie name, must contain only [0-9a-z_-] characters
|
| 'sess_expiration'
|
|     The number of SECONDS you want the session to last.
|     Setting to 0 (zero) means expire when the browser is closed.
|
| 'sess_save_path'
|
|     The location to save sessions to, driver dependent.
|
|     For the 'files' driver, it's a path to a writable directory.
|     WARNING: Only absolute paths are supported!
|
|     For the 'database' driver, it's a table name.
|     Please read up the manual for the format with other session drivers.
|
|     IMPORTANT: You are REQUIRED to set a valid save path!
|
| 'sess_match_ip'
|
|     Whether to match the user's IP address when reading the session data.
|
| 'sess_time_to_update'
|
|     How many seconds between CI regenerating the session ID.
|
| 'sess_regenerate_destroy'
|
|     Whether to destroy session data associated with the old session ID
|     when auto-regenerating the session ID. When set to FALSE, the data
|     will be later deleted by the garbage collector.
|
| Other session cookie settings are shared with the rest of the application,
| except for 'cookie_prefix' and 'cookie_httponly', which are ignored here.

```

```

*/
$config['sess_driver'] = 'files';
$config['sess_cookie_name'] = 'ci_session';
$config['sess_expiration'] = 7200;
$config['sess_save_path'] = APPPATH . 'cache/session';
$config['sess_match_ip'] = FALSE;
$config['sess_time_to_update'] = 300;
$config['sess_regenerate_destroy'] = FALSE;

/*
|-----
| Cookie Related Variables
|-----
|
| 'cookie_prefix'    = Set a cookie name prefix if you need to avoid collisions
| 'cookie_domain'    = Set to .your-domain.com for site-wide cookies
| 'cookie_path'      = Typically will be a forward slash
| 'cookie_secure'    = Cookie will only be set if a secure HTTPS connection
exists.
| 'cookie_httponly'  = Cookie will only be accessible via HTTP(S) (no
javascript)
|
| Note: These settings (with the exception of 'cookie_prefix' and
|       'cookie_httponly') will also affect sessions.
|
*/
$config['cookie_prefix']    = '';
$config['cookie_domain']    = '';
$config['cookie_path']      = '/';
$config['cookie_secure']    = FALSE;
$config['cookie_httponly']  = FALSE;

/*
|-----
| Standardize newlines
|-----
|
| Determines whether to standardize newline characters in input data,
| meaning to replace \r\n, \r, \n occurrences with the PHP_EOL value.
|
| This is particularly useful for portability between UNIX-based OSes,
| (usually \n) and Windows (\r\n).
|
*/
$config['standardize_newlines'] = FALSE;

/*
|-----
| Global XSS Filtering
|-----
|
| Determines whether the XSS filter is always active when GET, POST or
| COOKIE data is encountered

```

```

|
| WARNING: This feature is DEPRECATED and currently available only
|         for backwards compatibility purposes!
|
| */
$config['global_xss_filtering'] = FALSE;

/*
|-----
| Cross Site Request Forgery
|-----
|
| Enables a CSRF cookie token to be set. When set to TRUE, token will be
| checked on a submitted form. If you are accepting user data, it is strongly
| recommended CSRF protection be enabled.
|
| 'csrf_token_name' = The token name
| 'csrf_cookie_name' = The cookie name
| 'csrf_expire' = The number in seconds the token should expire.
| 'csrf_regenerate' = Regenerate token on every submission
| 'csrf_exclude_uris' = Array of URIs which ignore CSRF checks
| */
$config['csrf_protection'] = FALSE;
$config['csrf_token_name'] = 'csrf_test_name';
$config['csrf_cookie_name'] = 'csrf_cookie_name';
$config['csrf_expire'] = 7200;
$config['csrf_regenerate'] = TRUE;
$config['csrf_exclude_uris'] = array();

/*
|-----
| Output Compression
|-----
|
| Enables Gzip output compression for faster page loads. When enabled,
| the output class will test whether your server supports Gzip.
| Even if it does, however, not all browsers support compression
| so enable only if you are reasonably sure your visitors can handle it.
|
| Only used if zlib.output_compression is turned off in your php.ini.
| Please do not use it together with httpd-level output compression.
|
| VERY IMPORTANT: If you are getting a blank page when compression is enabled
it
| means you are prematurely outputting something to your browser. It could
| even be a line of whitespace at the end of one of your scripts. For
| compression to work, nothing can be sent before the output buffer is called
| by the output class. Do not 'echo' any values with compression enabled.
|
| */
$config['compress_output'] = FALSE;

/*
|-----

```

```

| Master Time Reference
| -----
|
| Options are 'local' or any PHP supported timezone. This preference tells
| the system whether to use your server's local time as the master 'now'
| reference, or convert it to the configured one timezone. See the 'date
| helper' page of the user guide for information regarding date handling.
|
| */
$config['time_reference'] = 'local';

/*
| -----
| Rewrite PHP Short Tags
| -----
|
| If your PHP installation does not have short tag support enabled CI
| can rewrite the tags on-the-fly, enabling you to utilize that syntax
| in your view files. Options are TRUE or FALSE (boolean)
|
| Note: You need to have eval() enabled for this to work.
|
| */
$config['rewrite_short_tags'] = FALSE;

/*
| -----
| Reverse Proxy IPs
| -----
|
| If your server is behind a reverse proxy, you must whitelist the proxy
| IP addresses from which CodeIgniter should trust headers such as
| HTTP_X_FORWARDED_FOR and HTTP_CLIENT_IP in order to properly identify
| the visitor's IP address.
|
| You can use both an array or a comma-separated list of proxy addresses,
| as well as specifying whole subnets. Here are a few examples:
|
| Comma-separated: '10.0.1.200,192.168.5.0/24'
| Array:           array('10.0.1.200', '192.168.5.0/24')
|
| */
$config['proxy_ips'] = '';
$config['modules_locations'] = array(
    APPPATH.'modules/' => '../modules/',
);

```

database.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

/*
| -----
| DATABASE CONNECTIVITY SETTINGS

```

```

| -----
| This file will contain the settings needed to access your database.
|
| For complete instructions please consult the 'Database Connection'
| page of the User Guide.
|
| -----
| EXPLANATION OF VARIABLES
| -----
|
| ['dsn']      The full DSN string describe a connection to the database.
| ['hostname'] The hostname of your database server.
| ['username'] The username used to connect to the database
| ['password'] The password used to connect to the database
| ['database'] The name of the database you want to connect to
| ['dbdriver'] The database driver. e.g.: mysqli.
|
|      Currently supported:
|          cubrid, ibase, mssql, mysql, mysqli, oci8,
|          odbc, pdo, postgre, sqlite, sqlite3, sqlsrv
| ['dbprefix'] You can add an optional prefix, which will be added
|              to the table name when using the Query Builder class
| ['pconnect'] TRUE/FALSE - Whether to use a persistent connection
| ['db_debug'] TRUE/FALSE - Whether database errors should be displayed.
| ['cache_on'] TRUE/FALSE - Enables/disables query caching
| ['cachedir'] The path to the folder where cache files should be stored
| ['char_set'] The character set used in communicating with the database
| ['dbcollat'] The character collation used in communicating with the
database
|
|      NOTE: For MySQL and MySQLi databases, this setting is only
used
|
|      as a backup if your server is running PHP < 5.2.3 or MySQL <
5.0.7
|
|      (and in table creation queries made with DB Forge).
|      There is an incompatibility in PHP with
mysql_real_escape_string() which
|
|      can make your site vulnerable to SQL injection if you are
using a
|
|      multi-byte character set and are running versions lower than
these.
|
|      Sites using Latin-1 or UTF-8 database character set and
collation are unaffected.
| ['swap_pre'] A default table prefix that should be swapped with the
dbprefix
| ['encrypt']  Whether or not to use an encrypted connection.
|
|      'mysql' (deprecated), 'sqlsrv' and 'pdo/sqlsrv' drivers accept
TRUE/FALSE
|
|      'mysqli' and 'pdo/mysql' drivers accept an array with the
following options:
|
|          'ssl_key'    - Path to the private key file
|          'ssl_cert'   - Path to the public key certificate file
|          'ssl_ca'     - Path to the certificate authority file

```

```

|         'ssl_capath' - Path to a directory containing trusted CA
certificats in PEM format
|         'ssl_cipher' - List of *allowed* ciphers to be used for the
encryption, separated by colons (':')
|         'ssl_verify' - TRUE/FALSE; Whether verify the server
certificate or not ('mysqli' only)
|
|     ['compress'] Whether or not to use client compression (MySQL only)
|     ['stricton'] TRUE/FALSE - forces 'Strict Mode' connections
|                     - good for ensuring strict SQL while developing
|     ['ssl_options'] Used to set various SSL options that can be used when
making SSL connections.
|     ['failover'] array - A array with 0 or more data for connections if the
main should fail.
|     ['save_queries'] TRUE/FALSE - Whether to "save" all executed queries.
|
|         NOTE: Disabling this will also effectively disable both
|
|         $this->db->last_query() and profiling of DB queries.
|
|         When you run a query, with this setting set to TRUE (default),
|
|         CodeIgniter will store the SQL statement for debugging
purposes.
|
|         However, this may cause high memory usage, especially if you
run
|
|         a lot of SQL queries ... disable this to avoid that problem.
|
| The $active_group variable lets you choose which connection group to
| make active. By default there is only one group (the 'default' group).
|
| The $query_builder variables lets you determine whether or not to load
| the query builder class.
*/
$active_group = 'default';
$query_builder = TRUE;

$db['default'] = array(
    'dsn' => '',
    'hostname' => 'localhost',
    'username' => 'root',
    'password' => '',
    'database' => 'webpro',
    'dbdriver' => 'mysqli',
    'dbprefix' => '',
    'pconnect' => FALSE,
    'db_debug' => (ENVIRONMENT !== 'production'),
    'cache_on' => FALSE,
    'cachedir' => '',
    'char_set' => 'utf8',
    'dbcollat' => 'utf8_general_ci',
    'swap_pre' => '',
    'encrypt' => FALSE,
    'compress' => FALSE,
    'stricton' => FALSE,
    'failover' => array(),
    'save_queries' => TRUE

```

```
);
```

```
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

/*
| -----
| URI ROUTING
| -----
| This file lets you re-map URI requests to specific controller functions.
|
| Typically there is a one-to-one relationship between a URL string
| and its corresponding controller class/method. The segments in a
| URL normally follow this pattern:
|
|    example.com/class/method/id/
|
| In some instances, however, you may want to remap this relationship
| so that a different class/function is called than the one
| corresponding to the URL.
|
| Please see the user guide for complete details:
|
|    http://codeigniter.com/user_guide/general/routing.html
|
| -----
| RESERVED ROUTES
| -----
|
| There are three reserved routes:
|
|    $route['default_controller'] = 'welcome';
|
| This route indicates which controller class should be loaded if the
| URI contains no data. In the above example, the "welcome" class
| would be loaded.
|
|    $route['404_override'] = 'errors/page_missing';
|
| This route will tell the Router which controller/method to use if those
| provided in the URL cannot be matched to a valid route.
|
|    $route['translate_uri_dashes'] = FALSE;
|
| This is not exactly a route, but allows you to automatically route
| controller and method names that contain dashes. '-' isn't a valid
| class or method name character, so it requires translation.
| When you set this option to TRUE, it will replace ALL dashes in the
| controller and method URI segments.
|
| Examples: my-controller/index -> my_controller/index
|           my-controller/my-method -> my_controller/my_method
```



```

*/
$route['default_controller'] = 'auth';
$route['404_override'] = '';
$route['translate_uri_dashes'] = FALSE;

$route['login'] = 'auth';
$route['register'] = 'auth/register';
$route['logout'] = 'auth/logout';
$route['dashboard'] = 'dashboard';
$route['peserta'] = 'peserta';
$route['peserta/simpan'] = 'peserta/save';
$route['peserta/hapus/(:num)'] = 'peserta/delete/$1';

```

autoload.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

/*
| -----
| AUTO-LOADER
| -----
| This file specifies which systems should be loaded by default.
|
| In order to keep the framework as light-weight as possible only the
| absolute minimal resources are loaded by default. For example,
| the database is not connected to automatically since no assumption
| is made regarding whether you intend to use it. This file lets
| you globally define which systems you would like loaded with every
| request.
|
| -----
| Instructions
| -----
|
| These are the things you can load automatically:
|
| 1. Packages
| 2. Libraries
| 3. Drivers
| 4. Helper files
| 5. Custom config files
| 6. Language files
| 7. Models
|
*/

/*
| -----
| Auto-load Packages
| -----
| Prototype:
|

```

```

| $autoload['packages'] = array(APPPATH.'third_party', '/usr/local/shared');
|
| */
$autoload['packages'] = array();

/*
| -----
| Auto-load Libraries
| -----
| These are the classes located in system/libraries/ or your
| application/libraries/ directory, with the addition of the
| 'database' library, which is somewhat of a special case.
|
| Prototype:
|
| $autoload['libraries'] = array('database', 'email', 'session');
|
| You can also supply an alternative library name to be assigned
| in the controller:
|
| $autoload['libraries'] = array('user_agent' => 'ua');
| */
$autoload['libraries'] = array('database', 'session', 'form_validation',
'upload');

/*
| -----
| Auto-load Drivers
| -----
| These classes are located in system/libraries/ or in your
| application/libraries/ directory, but are also placed inside their
| own subdirectory and they extend the CI_Driver_Library class. They
| offer multiple interchangeable driver options.
|
| Prototype:
|
| $autoload['drivers'] = array('cache');
| */
$autoload['drivers'] = array();

/*
| -----
| Auto-load Helper Files
| -----
| Prototype:
|
| $autoload['helper'] = array('url', 'file');
| */
$autoload['helper'] = array('url', 'form');

/*
| -----
| Auto-load Config files

```

```

| -----
| Prototype:
|
|     $autoload['config'] = array('config1', 'config2');
|
| NOTE: This item is intended for use ONLY if you have created custom
| config files. Otherwise, leave it blank.
|
| */
$autoload['config'] = array();
|
| /*
| -----
| Auto-load Language files
| -----
| Prototype:
|
|     $autoload['language'] = array('lang1', 'lang2');
|
| NOTE: Do not include the "_lang" part of your file. For example
| "codeigniter_lang.php" would be referenced as array('codeigniter');
|
| */
$autoload['language'] = array();
|
| /*
| -----
| Auto-load Models
| -----
| Prototype:
|
|     $autoload['model'] = array('first_model', 'second_model');
|
| You can also supply an alternative model name to be assigned
| in the controller:
|
|     $autoload['model'] = array('first_model' => 'first');
|
| */
$autoload['model'] = array();

```

Controller

Auth.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

// Controller ini menangani alur autentikasi pengguna.
class Auth extends CI_Controller
{
    public function __construct()
    {
        parent::__construct();
    }
}

```

```

        // Memuat model user untuk proses login dan register.
        $this->load->model('User_model');
    }

    public function index()
    {
        // Jika user sudah login, arahkan langsung ke dashboard.
        if ($this->session->userdata('user_id')) {
            redirect('dashboard');
        }

        $data['title'] = 'Login | InklusSkill';

        if ($this->input->method() === 'post') {
            // Validasi input login sebelum dicek ke database.
            $this->form_validation->set_rules('email', 'Email',
            'required|trim|valid_email');
            $this->form_validation->set_rules('password', 'Password',
            'required|trim');

            if ($this->form_validation->run()) {
                // Cek kecocokan email dan password melalui model.
                $user = $this->User_model->authenticate(
                    $this->input->post('email', true),
                    $this->input->post('password', false)
                );

                if ($user) {
                    // Simpan data penting user ke session saat login
                    berhasil.

                    $this->session->set_userdata(array(
                        'user_id' => $user->id,
                        'username' => $user->username,
                        'email' => $user->email,
                        'role' => $user->role,
                        'logged_in' => true,
                    ));

                    $this->session->set_flashdata('success', 'Login berhasil.
                    Selamat datang di dashboard.');
```

```

                    redirect('dashboard');
                }

                $this->session->set_flashdata('error', 'Email atau password
                tidak cocok.');
```

```

                    redirect('login');
                }
            }

            $this->load->view('auth/login', $data);
        }

        public function register()

```

```

{
    // User yang sudah login tidak perlu membuka halaman register lagi.
    if ($this->session->userdata('user_id')) {
        redirect('dashboard');
    }

    $data['title'] = 'Register | InklusSkill';

    if ($this->input->method() === 'post') {
        // Validasi data register sebelum disimpan ke database.
        $this->form_validation->set_rules('username', 'Nama Lengkap',
        'required|trim|min_length[3]');
        $this->form_validation->set_rules('email', 'Email',
        'required|trim|valid_email|is_unique[ci3_users.email]');
        $this->form_validation->set_rules('password', 'Password',
        'required|trim|min_length[5]');

        if ($this->form_validation->run()) {
            // Password disimpan dalam bentuk hash agar lebih aman.
            $payload = array(
                'username' => $this->input->post('username', true),
                'email' => $this->input->post('email', true),
                'password' => password_hash($this->input->post('password',
false), PASSWORD_DEFAULT),
                'role' => 'sekolah',
            );

            $this->User_model->create($payload);
            $this->session->set_flashdata('success', 'Registrasi berhasil.
Silakan login.');
```

Dashboard.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

// Controller dashboard hanya bisa diakses oleh user yang sudah login.
class Dashboard extends CI_Controller
{

```

```

public function __construct()
{
    parent::__construct();
    // Pastikan akses dashboard terlindungi oleh session login.
    $this->require_login();
    $this->load->model('Participant_model');
}

public function index()
{
    // Ambil data peserta dan statistik untuk ditampilkan di dashboard.
    $participants = $this->Participant_model->get_all();
    $stats = $this->Participant_model->get_stats();

    $data = array(
        'title' => 'Dashboard | InklusSkill',
        'participants' => $participants,
        'stats' => $stats,
    );

    $this->load->view('dashboard/index', $data);
}

private function require_login()
{
    // Redirect ke halaman login jika session user tidak ditemukan.
    if (!$this->session->userdata('user_id')) {
        $this->session->set_flashdata('error', 'Silakan login terlebih
dahulu.');
```

Peserta.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

// Controller ini mengatur CRUD peserta dan proses upload foto.
class Peserta extends CI_Controller
{
    // Folder tujuan penyimpanan file upload.
    private $upload_path;
    // Batas ukuran upload foto dalam KB.
    private $max_upload_size_kb = 4096;

    public function __construct()
    {
        parent::__construct();
        // Halaman peserta hanya bisa diakses setelah login.
        $this->require_login();
        $this->load->model('Participant_model');
```

```

        $this->upload_path = FCPATH . 'uploads/';
    }

    public function index()
    {
        // Ambil data peserta dan daftar pelatihan untuk halaman kelola
peserta.
        $data = array(
            'title' => 'Kelola Peserta | InkluSkill',
            'participants' => $this->Participant_model->get_all(),
            'trainings' => $this->Participant_model->training_options(),
        );

        $this->load->view('participants/index', $data);
    }

    public function save()
    {
        // Ambil id untuk membedakan proses tambah dan edit.
        $id = (int) $this->input->post('id');
        $old_photo = $this->input->post('foto_lama', true);

        // Validasi field utama form peserta.
        $this->form_validation->set_rules('nama', 'Nama Peserta',
'required|trim');
        $this->form_validation->set_rules('umur', 'Usia',
'required|integer|greater_than_equal_to[1]');
        $this->form_validation->set_rules('jenis_kelamin', 'Jenis Kelamin',
'required|trim');
        $this->form_validation->set_rules('pelatihan', 'Pelatihan',
'required|trim');

        if (!$this->form_validation->run()) {
            $this->session->set_flashdata('error', validation_errors('<div>',
'</div>'));
            redirect('peserta');
        }

        // Susun data peserta yang akan disimpan ke database.
        $payload = array(
            'nama' => $this->input->post('nama', true),
            'umur' => (int) $this->input->post('umur', true),
            'jenis_kelamin' => $this->input->post('jenis_kelamin', true),
            'pelatihan' => $this->input->post('pelatihan', true),
        );

        $uploaded_photo = $old_photo;
        $has_new_photo = !empty($_FILES['foto']['name']);

        if ($has_new_photo) {
            // Upload file foto baru jika user memilih file.
            $result = $this->handle_upload();
        }
    }

```

```

        if (!$result['status']) {
            $this->session->set_flashdata('error', $result['message']);
            redirect('peserta');
        }

        $uploaded_photo = $result['file_name'];
    } elseif ($id === 0) {
        // Saat tambah data baru, foto wajib diisi.
        $this->session->set_flashdata('error', 'Foto peserta wajib
diupload saat tambah data.');
```

diupload saat tambah data.');

```

        redirect('peserta');
    }

    $payload['foto'] = $uploaded_photo;

    if ($id > 0) {
        // Update data peserta jika id sudah ada.
        $this->Participant_model->update($id, $payload);

        if ($has_new_photo && $old_photo && $old_photo !==
$uploaded_photo) {
            // Hapus foto lama jika diganti dengan file baru.
            $this->delete_photo_file($old_photo);
        }

        $message = 'Data peserta berhasil diperbarui.';
    } else {
        // Tambah data peserta baru ke database.
        $this->Participant_model->create($payload);
        $message = 'Data peserta berhasil ditambahkan.';
    }

    $this->session->set_flashdata('success', $message);
    redirect('peserta');
}

public function delete($id)
{
    // Ambil data peserta untuk memastikan record tersedia.
    $participant = $this->Participant_model->find($id);

    if (!$participant) {
        $this->session->set_flashdata('error', 'Data peserta tidak
ditemukan.');
```

ditemukan.');

```

        redirect('peserta');
    }

    // Hapus data peserta dan file fotonya.
    $this->Participant_model->delete($id);
    $this->delete_photo_file($participant->foto);

    $this->session->set_flashdata('success', 'Data peserta berhasil
dihapus.');
```



```

        redirect('peserta');
    }

    private function handle_upload()
    {
        // Konfigurasi upload file gambar peserta.
        $config['upload_path'] = $this->upload_path;
        $config['allowed_types'] = 'jpg|jpeg|png|gif|webp';
        $config['max_size'] = $this->max_upload_size_kb;
        $config['encrypt_name'] = true;

        if (!is_dir($config['upload_path'])) {
            // Buat folder upload jika belum tersedia.
            mkdir($config['upload_path'], 0777, true);
        }

        $this->upload->initialize($config);

        if (!$this->upload->do_upload('foto')) {
            // Terjemahkan pesan error bawaan upload ke bahasa Indonesia.
            $raw_error = strip_tags($this->upload->display_errors('', ''));

            return array(
                'status' => false,
                'message' => $this->translate_upload_error($raw_error),
            );
        }

        $upload_data = $this->upload->data();

        return array(
            'status' => true,
            // Simpan nama file hasil upload untuk dicatat ke database.
            'file_name' => $upload_data['file_name'],
        );
    }

    private function translate_upload_error($message)
    {
        // Ubah batas upload dari KB ke MB agar mudah dibaca user.
        $max_size_mb = $this->max_upload_size_kb / 1024;
        $formatted_size = rtrim(rtrim(number_format($max_size_mb, 2, '.', ''),
'0'), '.');

        if (strpos($message, 'larger than the permitted size') !== false) {
            return 'Ukuran foto melebihi batas maksimal ' . $formatted_size .
' MB.';
        }

        if (strpos($message, 'filetype you are attempting to upload is not
allowed') !== false) {
            return 'Format foto tidak didukung. Gunakan file jpg, jpeg, png,
gif, atau webp.';
        }
    }

```

```

    }

    if (stripos($message, 'did not select a file to upload') !== false) {
        return 'Silakan pilih file foto terlebih dahulu.';
    }

    return 'Upload foto gagal. ' . $message;
}

private function delete_photo_file($filename)
{
    if (!$filename) {
        return;
    }

    // Hapus file fisik dari folder uploads jika file ditemukan.
    $path = $this->upload_path . $filename;
    if (is_file($path)) {
        unlink($path);
    }
}

private function require_login()
{
    // Cegah akses langsung ke halaman peserta tanpa login.
    if (!$this->session->userdata('user_id')) {
        $this->session->set_flashdata('error', 'Silakan login terlebih
dahulu. ');
        redirect('login');
    }
}
}

```

Model

User_model.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

// Model ini menangani data pengguna untuk autentikasi.
class User_model extends CI_Model
{
    private $table = 'ci3_users';

    public function authenticate($email, $password)
    {
        // Ambil user berdasarkan email terlebih dahulu.
        $user = $this->db->get_where($this->table, array('email' => $email))-
>row();

        if (!$user) {
            return null;
        }
    }
}

```

```

    }

    // Verifikasi password hash sebelum login dinyatakan berhasil.
    return password_verify($password, $user->password) ? $user : null;
}

public function create($data)
{
    // Simpan akun baru ke tabel user.
    return $this->db->insert($this->table, $data);
}
}

```

Participant_model.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

// Model ini menangani operasi database untuk data peserta.
class Participant_model extends CI_Model
{
    private $table = 'ci3_participants';

    public function get_all()
    {
        // Ambil seluruh peserta dan urutkan dari data terbaru.
        return $this->db->order_by('id', 'DESC')->get($this->table)->result();
    }

    public function find($id)
    {
        // Ambil satu data peserta berdasarkan id.
        return $this->db->get_where($this->table, array('id' => (int) $id))->row();
    }

    public function create($data)
    {
        // Tambahkan data peserta baru.
        return $this->db->insert($this->table, $data);
    }

    public function update($id, $data)
    {
        // Perbarui data peserta berdasarkan id.
        return $this->db->where('id', (int) $id)->update($this->table, $data);
    }

    public function delete($id)
    {
        // Hapus data peserta berdasarkan id.
        return $this->db->where('id', (int) $id)->delete($this->table);
    }
}

```

```

    public function get_stats()
    {
        // Hitung ringkasan data untuk dashboard.
        $participants = $this->db->count_all($this->table);
        $female = $this->db->where('jenis_kelamin', 'Perempuan')->count_all_results($this->table);
        $male = $this->db->where('jenis_kelamin', 'Laki-laki')->count_all_results($this->table);

        return array(
            'total' => $participants,
            'female' => $female,
            'male' => $male,
            'training_count' => count($this->training_options()),
        );
    }

    public function training_options()
    {
        // Daftar pelatihan default yang ditampilkan di form peserta.
        return array('Moshing', 'Icikiwir', 'Desain Grafis', 'Menjahit', 'Public Speaking');
    }
}

```

View

login.php

```

<?php
// Memanggil partial head untuk elemen <head> dan asset utama.
$this->load->view('partials/head', array('title' => $title));
?>
<main class="auth-body">
    <section class="auth-container">
        <div class="auth-copy">
            <p class="auth-badge">CodeIgniter 3</p>
            <h1>Masuk ke InkluSkill</h1>
            <p class="auth-subtitle">Dashboard hanya bisa diakses setelah login berhasil, sesuai instruksi modul.</p>
        </div>

        <?php
        // Menampilkan notifikasi error atau sukses jika ada.
        $this->load->view('partials/alerts');
        ?>

        <?php
        // Form login mengirim data ke route login.
        echo form_open('login', array('class' => 'auth-form'));
        ?>
        <label>Email</label>

```

```

        <input type="email" name="email" value="<?php echo
set_value('email'); ?>" placeholder="Masukkan email" required>

        <label>Password</label>
        <input type="password" name="password" placeholder="Masukkan
password" required>

        <button type="submit" class="btn-primary">Login</button>
        <?php echo form_close(); ?>

        <p class="auth-switch">Belum punya akun? <a href="<?php echo
site_url('register'); ?>">Daftar</a></p>
    </section>
</main>
<?php
// Memanggil partial footer untuk script JS penutup.
$this->load->view('partials/footer');
?>

```

register.php

```

<?php
// Memanggil partial head untuk kebutuhan tampilan halaman register.
$this->load->view('partials/head', array('title' => $title));
?>
<main class="auth-body">
    <section class="auth-container">
        <div class="auth-copy">
            <p class="auth-badge">Registrasi</p>
            <h1>Buat akun baru</h1>
            <p class="auth-subtitle">Akun ini dipakai untuk masuk ke dashboard
utama aplikasi.</p>
        </div>

        <?php
        // Menampilkan pesan validasi atau status proses register.
        $this->load->view('partials/alerts');
        ?>

        <?php
        // Form register mengirim data akun baru ke controller Auth.
        echo form_open('register', array('class' => 'auth-form'));
        ?>

        <label>Nama Lengkap</label>
        <input type="text" name="username" value="<?php echo
set_value('username'); ?>" placeholder="Masukkan nama lengkap" required>

        <label>Email</label>
        <input type="email" name="email" value="<?php echo
set_value('email'); ?>" placeholder="Masukkan email" required>

        <label>Password</label>

```

```

        <input type="password" name="password" placeholder="Minimal 5
karakter" required>

        <button type="submit" class="btn-primary">Daftar</button>
    <?php echo form_close(); ?>

    <p class="auth-switch">Sudah punya akun? <a href="<?php echo
site_url('login'); ?>">Login</a></p>
</section>
</main>
<?php
// Memanggil partial footer untuk menutup struktur halaman.
$this->load->view('partials/footer');
?>

```

dashboard/index.php

```

<?php
// Head berisi title, font, icon, dan stylesheet utama aplikasi.
$this->load->view('partials/head', array('title' => $title));
?>
<div class="app-shell">
    <?php
    // Sidebar menampilkan menu navigasi utama aplikasi.
    $this->load->view('partials/sidebar');
    ?>

    <main class="main-content">
        <header class="page-header">
            <div>
                <p class="eyebrow">Dashboard</p>
                <h1>Ringkasan aplikasi peserta pelatihan</h1>
                <p class="header-subtitle">Halaman ini hanya muncul setelah
login berhasil.</p>
            </div>
            <div class="profile-chip">
                <span class="profile-avatar"><?php echo
strtoupper(substr($this->session->userdata('username'), 0, 1)); ?></span>
                <div>
                    <strong><?php echo html_escape($this->session-
>userdata('username')); ?></strong>
                    <small><?php echo html_escape($this->session-
>userdata('email')); ?></small>
                </div>
            </div>
        </header>

        <?php
        // Menampilkan notifikasi session jika ada.
        $this->load->view('partials/alerts');
        ?>

        <section class="stats-grid">

```

```

<article class="stat-card">
    <span class="stat-icon"><i class="ri-group-line"></i></span>
    <h2><?php echo $stats['total']; ?></h2>
    <p>Total peserta terdaftar</p>
</article>
<article class="stat-card">
    <span class="stat-icon"><i class="ri-women-line"></i></span>
    <h2><?php echo $stats['female']; ?></h2>
    <p>Peserta perempuan</p>
</article>
<article class="stat-card">
    <span class="stat-icon"><i class="ri-men-line"></i></span>
    <h2><?php echo $stats['male']; ?></h2>
    <p>Peserta laki-laki</p>
</article>
<article class="stat-card">
    <span class="stat-icon"><i class="ri-book-open-
line"></i></span>
    <h2><?php echo $stats['training_count']; ?></h2>
    <p>Jenis pelatihan aktif</p>
</article>
</section>

<section class="panel">
    <div class="panel-heading">
        <div>
            <h2>Peserta terbaru</h2>
            <p>Ringkasan data dari fitur CRUD peserta.</p>
        </div>
        <a href="<?php echo site_url('peserta'); ?>" class="btn-
primary">Kelola Peserta</a>
    </div>

    <div class="table-wrap">
        <table>
            <thead>
                <tr>
                    <th>No</th>
                    <th>Nama</th>
                    <th>Usia</th>
                    <th>Jenis Kelamin</th>
                    <th>Pelatihan</th>
                </tr>
            </thead>
            <tbody>
                <?php if ($participants): ?>
                    <?php foreach (array_slice($participants, 0, 5) as
$index => $participant): ?>
                        <tr>
                            <td><?php echo $index + 1; ?></td>
                            <td><?php echo html_escape($participant-
>nama); ?></td>

```

```

                                <td><?php echo (int) $participant->umur;
?></td>

                                <td><?php echo html_escape($participant-
>jenis_kelamin); ?></td>

                                <td><?php echo html_escape($participant-
>pelatihan); ?></td>

                                </tr>
                                <?php endforeach; ?>
                                <?php else: ?>
                                <tr>
                                <td colspan="5">Belum ada peserta yang
tersimpan.</td>

                                </tr>
                                <?php endif; ?>
                                </tbody>
                                </table>
                                </div>
                                </section>
                                </main>
</div>
<?php
// Footer memuat script yang diperlukan di akhir halaman.
$this->load->view('partials/footer');
?>

```

participants/index.php

```

<?php
// Partial head dipakai ulang agar struktur halaman konsisten.
$this->load->view('partials/head', array('title' => $title));
?>
<div class="app-shell">
    <?php
    // Sidebar berisi menu dashboard, peserta, dan logout.
    $this->load->view('partials/sidebar');
    ?>

    <main class="main-content">
        <header class="page-header">
            <div>
                <p class="eyebrow">CRUD Peserta</p>
                <h1>Kelola peserta pelatihan</h1>
                <p class="header-subtitle">Tombol <strong>Simpan</strong>
menangani create/update dan upload foto dalam satu submit.</p>
            </div>
            <button type="button" class="btn-primary" id="openModal">
                <i class="ri-add-circle-line"></i> Tambah Peserta
            </button>
        </header>

        <?php
        // Menampilkan pesan proses CRUD dan upload foto.
        $this->load->view('partials/alerts');
    </main>
</div>

```



```

?>

<section class="panel">
    <div class="panel-heading">
        <div>
            <h2>Data peserta</h2>
            <p>Cari, tambah, ubah, dan hapus data peserta dari satu
halaman.</p>
        </div>
        <input type="text" id="searchPeserta" class="search-box"
placeholder="Cari nama peserta...">
        </div>

        <div class="table-wrap">
            <table>
                <thead>
                    <tr>
                        <th>No</th>
                        <th>Nama</th>
                        <th>Usia</th>
                        <th>Jenis Kelamin</th>
                        <th>Pelatihan</th>
                        <th>Foto</th>
                        <th>Aksi</th>
                    </tr>
                </thead>
                <tbody id="pesertaTable">
                    <?php if ($participants): ?>
                        <?php foreach ($participants as $index =>
$participant): ?>
                            <tr>
                                <td><?php echo $index + 1; ?></td>
                                <td><?php echo html_escape($participant-
>nama); ?></td>
                                <td><?php echo (int) $participant->umur;
?></td>
                                <td><?php echo html_escape($participant-
>jenis_kelamin); ?></td>
                                <td><?php echo html_escape($participant-
>pelatihan); ?></td>
                                <td>
                                    
                                </td>
                                <td class="action-cell">
                                    <button
                                        type="button"
                                        class="btn-soft"
                                        data-id="<?php echo $participant-
>id; ?>"
                                        data-nama="<?php echo
html_escape($participant->nama); ?>"

```

```

data-umur="<?php echo (int)
$participant->umur; ?>"
data-jk="<?php echo
html_escape($participant->jenis_kelamin); ?>"
data-pelatihan="<?php echo
html_escape($participant->pelatihan); ?>"
data-foto="<?php echo
html_escape($participant->foto); ?>"
onclick="editPeserta(this)"
>
Edit
</button>
<a href="<?php echo
site_url('peserta/hapus/' . $participant->id); ?>" class="btn-danger"
onclick="return confirm('Hapus data peserta ini?');">Hapus</a>
</td>
</tr>
<?php endforeach; ?>
<?php else: ?>
<tr>
<td colspan="7">Belum ada data peserta.</td>
</tr>
<?php endif; ?>
</tbody>
</table>
</div>
</section>
</main>
</div>

<div class="modal" id="modalForm">
<div class="modal-content">
<div class="modal-header">
<div>
<p class="eyebrow">Form Peserta</p>
<h3 id="modalTitle">Tambah Peserta</h3>
</div>
<button type="button" class="icon-button" id="closeModal"><i
class="ri-close-line"></i></button>
</div>

<?php
// Form multipart dipakai agar data peserta dan file foto bisa dikirim
bersama.
echo form_open_multipart('peserta/simpan', array('id' =>
'pesertaForm'));
?>

<input type="hidden" name="id" id="idInput" value="">
<input type="hidden" name="foto_lama" id="fotoLamaInput" value="">

<label>Nama Peserta</label>
<input type="text" name="nama" id="namaInput" required>

```

```

        <label>Usia</label>
        <input type="number" name="umur" id="umurInput" min="1" required>

        <label>Jenis Kelamin</label>
        <select name="jenis_kelamin" id="genderInput" required>
            <option value="Laki-laki">Laki-laki</option>
            <option value="Perempuan">Perempuan</option>
        </select>

        <label>Pelatihan</label>
        <select name="pelatihan" id="pelatihanInput" required>
            <?php foreach ($trainings as $training): ?>
                <option value="<?php echo html_escape($training);
?>"><?php echo html_escape($training); ?></option>
            <?php endforeach; ?>
        </select>

        <label>Upload Foto</label>
        <input type="file" name="foto" id="fotoInput"
accept=".jpg,.jpeg,.png,.gif,.webp">
        <p id="fotoInfo" class="foto-info">Foto peserta wajib saat tambah
data.</p>

        <div class="modal-buttons">
            <button type="submit" class="btn-primary">Simpan</button>
            <button type="button" class="btn-secondary"
id="cancelModal">Batal</button>
        </div>
        <?php echo form_close(); ?>
    </div>
</div>
<?php
// Footer menutup halaman dan memanggil JavaScript peserta.
$this->load->view('partials/footer');
?>

```

alerts.php

```

<?php
// Partial ini dipakai untuk menampilkan notifikasi sukses, error, dan
validasi form.
?>
<?php if ($this->session->flashdata('success')): ?>
    <div class="alert alert-success"><?php echo $this->session-
>flashdata('success'); ?></div>
<?php endif; ?>

<?php if ($this->session->flashdata('error')): ?>
    <div class="alert alert-error"><?php echo $this->session-
>flashdata('error'); ?></div>
<?php endif; ?>

<?php if (validation_errors()): ?>

```

```

    <div class="alert alert-error"><?php echo validation_errors(); ?></div>
<?php endif; ?>

```

sidebar.php

```

<?php
// Sidebar navigasi ditampilkan di halaman dashboard dan peserta.
?>
<aside class="sidebar">
    <h2 class="sidebar-title">InkluSkill</h2>
    <a class="<?php echo current_url() === site_url('dashboard') ? 'active' :
'; ?>" href="<?php echo site_url('dashboard'); ?>">
        <i class="ri-dashboard-line"></i> Dashboard
    </a>
    <a class="<?php echo current_url() === site_url('peserta') ? 'active' :
'; ?>" href="<?php echo site_url('peserta'); ?>">
        <i class="ri-user-3-line"></i> Kelola Peserta
    </a>
    <a href="<?php echo site_url('logout'); ?>" class="logout-link">
        <i class="ri-logout-box-line"></i> Logout
    </a>
</aside>

```

head.php

```

<?php
// Partial head berisi metadata halaman dan asset CSS utama aplikasi.
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><?php echo isset($title) ? $title : 'InkluSkill'; ?></title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap" rel="stylesheet">
    <link
href="https://cdn.jsdelivr.net/npm/remixicon@4.2.0/fonts/remixicon.css"
rel="stylesheet">
    <link rel="stylesheet" href="<?php echo base_url('assets/app.css'); ?>">
</head>
<body>

```

footer.php

```

<?php
// Partial footer memanggil file JavaScript sebelum penutup body.
?>
<script src="<?php echo base_url('assets/peserta.js'); ?>"></script>
</body>

```

```
</html>
```

Asset

app.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: 'Poppins', sans-serif;
}

body {
  background: linear-gradient(180deg, #f4f7fb 0%, #eef3ff 100%);
  color: #1f2937;
}

a {
  text-decoration: none;
}

.auth-body {
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 24px;
}

.auth-container {
  width: min(100%, 460px);
  background: #ffffff;
  border-radius: 24px;
  padding: 32px;
  box-shadow: 0 24px 60px rgba(68, 97, 215, 0.14);
}

.auth-copy h1 {
  font-size: 2rem;
  color: #243b8f;
  margin-bottom: 8px;
}

.auth-subtitle {
  color: #6b7280;
  line-height: 1.6;
  margin-bottom: 20px;
}

.auth-badge,
.eyebrow {
  display: inline-flex;
  align-items: center;
```

```
    gap: 8px;
    padding: 8px 12px;
    border-radius: 999px;
    background: #e8efff;
    color: #4461d7;
    font-size: 0.85rem;
    font-weight: 600;
    margin-bottom: 14px;
}

.auth-form,
.modal-content form {
    display: grid;
    gap: 12px;
}

.auth-form label,
.modal-content label {
    font-weight: 600;
    color: #374151;
}

.auth-form input,
.modal-content input,
.modal-content select,
.search-box {
    width: 100%;
    border: 1px solid #dbe3f3;
    border-radius: 14px;
    padding: 14px 16px;
    outline: none;
    background: #fbfdff;
}

.btn-primary,
.btn-secondary,
.btn-soft,
.btn-danger {
    border: none;
    border-radius: 14px;
    padding: 12px 18px;
    cursor: pointer;
    font-weight: 600;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
}

.btn-primary {
    background: #4461d7;
    color: #fff;
}
```

```
.btn-secondary {
  background: #e5e7eb;
  color: #111827;
}

.btn-soft {
  background: #e8efff;
  color: #4461d7;
}

.btn-danger {
  background: #fee2e2;
  color: #dc2626;
}

.auth-switch {
  text-align: center;
  margin-top: 18px;
  color: #6b7280;
}

.auth-switch a {
  color: #4461d7;
  font-weight: 600;
}

.app-shell {
  display: flex;
  min-height: 100vh;
}

.sidebar {
  width: 250px;
  background: #ffffff;
  border-right: 1px solid #e5e7eb;
  padding: 28px 20px;
}

.sidebar-title {
  color: #243b8f;
  font-size: 1.5rem;
  margin-bottom: 24px;
}

.sidebar a {
  display: block;
  padding: 14px 16px;
  border-radius: 14px;
  margin-bottom: 8px;
  color: #374151;
}
```

```
.sidebar a.active,
.sidebar a:hover {
    background: #e8efff;
    color: #4461d7;
}

.logout-link {
    margin-top: 24px;
}

.main-content {
    flex: 1;
    padding: 32px;
}

.page-header {
    display: flex;
    justify-content: space-between;
    gap: 20px;
    align-items: flex-start;
    margin-bottom: 24px;
}

.page-header h1 {
    font-size: 2rem;
    color: #1f2a44;
    margin-bottom: 8px;
}

.header-subtitle {
    color: #6b7280;
}

.profile-chip {
    display: flex;
    align-items: center;
    gap: 12px;
    background: #fff;
    border-radius: 18px;
    padding: 12px 16px;
    box-shadow: 0 12px 30px rgba(31, 42, 68, 0.08);
}

.profile-avatar {
    width: 42px;
    height: 42px;
    display: inline-grid;
    place-items: center;
    border-radius: 50%;
    background: #4461d7;
    color: #fff;
    font-weight: 700;
}
```



```
.profile-chip small {
  display: block;
  color: #6b7280;
}

.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
  margin-bottom: 24px;
}

.stat-card,
.panel {
  background: #fff;
  border-radius: 24px;
  padding: 24px;
  box-shadow: 0 18px 45px rgba(31, 42, 68, 0.08);
}

.stat-card h2 {
  font-size: 2.2rem;
  color: #243b8f;
  margin: 12px 0 8px;
}

.stat-card p {
  color: #6b7280;
}

.stat-icon {
  width: 52px;
  height: 52px;
  display: inline-grid;
  place-items: center;
  border-radius: 16px;
  background: #e8efff;
  color: #4461d7;
  font-size: 1.4rem;
}

.panel-heading {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
  margin-bottom: 18px;
}

.panel-heading h2 {
  margin-bottom: 6px;
}
```

```
.panel-heading p {
  color: #6b7280;
}

.table-wrap {
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  padding: 14px 12px;
  text-align: left;
  border-bottom: 1px solid #edf2f7;
  vertical-align: middle;
}

th {
  background: #f6f8ff;
  color: #243b8f;
}

.thumb {
  width: 70px;
  height: 70px;
  object-fit: cover;
  border-radius: 16px;
}

.action-cell {
  display: flex;
  gap: 10px;
}

.modal {
  position: fixed;
  inset: 0;
  background: rgba(17, 24, 39, 0.5);
  display: none;
  align-items: center;
  justify-content: center;
  padding: 20px;
}

.modal.show {
  display: flex;
}
```

```
.modal-content {
  width: min(100%, 460px);
  background: #fff;
  border-radius: 24px;
  padding: 24px;
}

.modal-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  margin-bottom: 16px;
}

.icon-button {
  width: 40px;
  height: 40px;
  border-radius: 50%;
  border: none;
  background: #f3f4f6;
  cursor: pointer;
}

.modal-buttons {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
  margin-top: 8px;
}

.foto-info {
  color: #6b7280;
  font-size: 0.9rem;
}

.alert {
  padding: 14px 16px;
  border-radius: 14px;
  margin-bottom: 18px;
  line-height: 1.6;
}

.alert-success {
  background: #dcfce7;
  color: #166534;
}

.alert-error {
  background: #fee2e2;
  color: #991b1b;
}

@media (max-width: 900px) {
```

```

    .app-shell {
        flex-direction: column;
    }

    .sidebar {
        width: 100%;
        border-right: 0;
        border-bottom: 1px solid #e5e7eb;
    }

    .page-header,
    .panel-heading {
        flex-direction: column;
        align-items: stretch;
    }

    .main-content {
        padding: 20px;
    }
}

```

peserta.js

```

const modal = document.getElementById('modalForm');
const openModalButton = document.getElementById('openModal');
const closeModalButton = document.getElementById('closeModal');
const cancelModalButton = document.getElementById('cancelModal');
const searchInput = document.getElementById('searchPeserta');
const form = document.getElementById('pesertaForm');
const modalTitle = document.getElementById('modalTitle');
const idInput = document.getElementById('idInput');
const fotoLamaInput = document.getElementById('fotoLamaInput');
const namaInput = document.getElementById('namaInput');
const umurInput = document.getElementById('umurInput');
const genderInput = document.getElementById('genderInput');
const pelatihanInput = document.getElementById('pelatihanInput');
const fotoInput = document.getElementById('fotoInput');
const fotoInfo = document.getElementById('fotoInfo');

function resetForm() {
    if (!form) {
        return;
    }

    form.reset();
    idInput.value = '';
    fotoLamaInput.value = '';
    modalTitle.textContent = 'Tambah Peserta';
    fotoInput.required = true;
    fotoInfo.textContent = 'Foto peserta wajib saat tambah data.';
}

function openModal() {

```

```

    if (modal) {
        modal.classList.add('show');
    }
}

function closeModal() {
    if (modal) {
        modal.classList.remove('show');
    }
}

if (openModalButton) {
    openModalButton.addEventListener('click', () => {
        resetForm();
        openModal();
    });
}

if (closeModalButton) {
    closeModalButton.addEventListener('click', closeModal);
}

if (cancelModalButton) {
    cancelModalButton.addEventListener('click', closeModal);
}

if (modal) {
    modal.addEventListener('click', (event) => {
        if (event.target === modal) {
            closeModal();
        }
    });
}

window.editPeserta = function (button) {
    idInput.value = button.dataset.id || '';
    fotoLamaInput.value = button.dataset.foto || '';
    namaInput.value = button.dataset.nama || '';
    umurInput.value = button.dataset.umur || '';
    genderInput.value = button.dataset.jk || 'Laki-laki';
    pelatihanInput.value = button.dataset.pelatihan || '';
    fotoInput.value = '';
    fotoInput.required = false;
    modalTitle.textContent = 'Edit Peserta';
    fotoInfo.textContent = button.dataset.foto
        ? `Foto saat ini: ${button.dataset.foto}. Upload lagi jika ingin
mengganti.`
        : 'Belum ada foto. Upload foto baru untuk melengkapinya.';
    openModal();
};

if (searchInput) {
    searchInput.addEventListener('keyup', function () {

```

```

    const keyword = this.value.toLowerCase();
    const rows = document.querySelectorAll('#pesertaTable tr');

    rows.forEach((row) => {
        const targetCell = row.children[1];
        if (!targetCell) {
            return;
        }

        row.style.display =
targetCell.textContent.toLowerCase().includes(keyword) ? '' : 'none';
    });
});
}

```

Database

webpro_ci3.sql

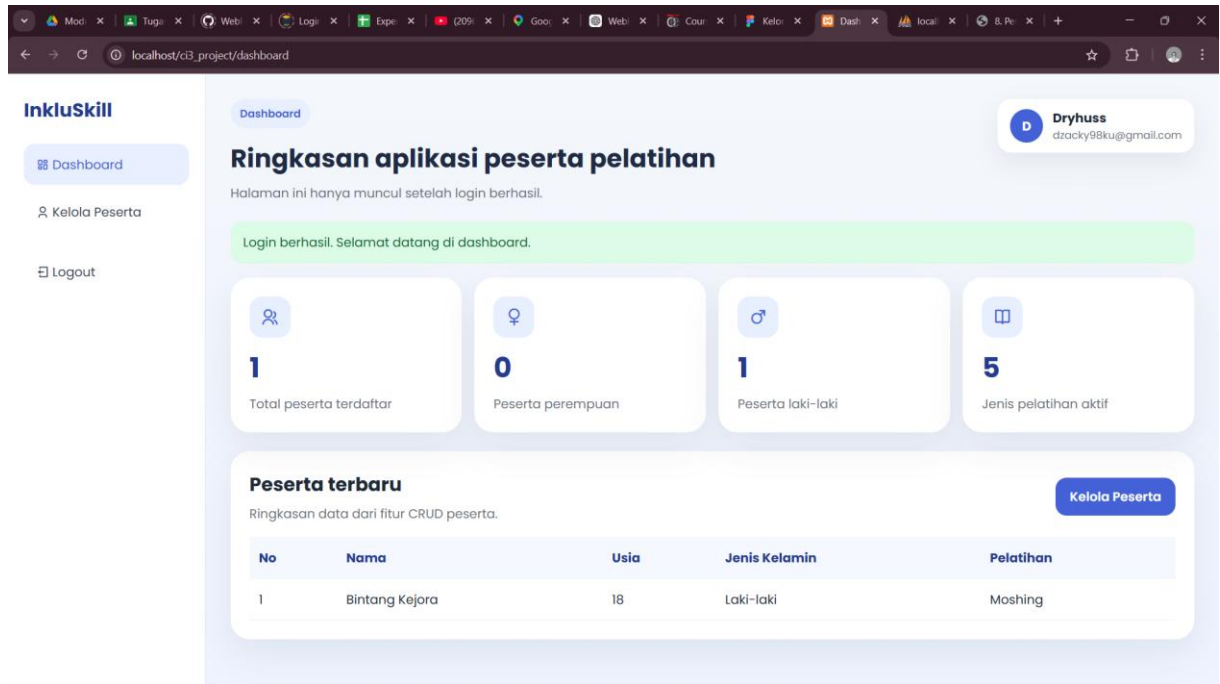
```

CREATE TABLE IF NOT EXISTS `ci3_users` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `username` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `password` varchar(255) NOT NULL,
  `role` varchar(50) NOT NULL DEFAULT 'sekolah',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  UNIQUE KEY `email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

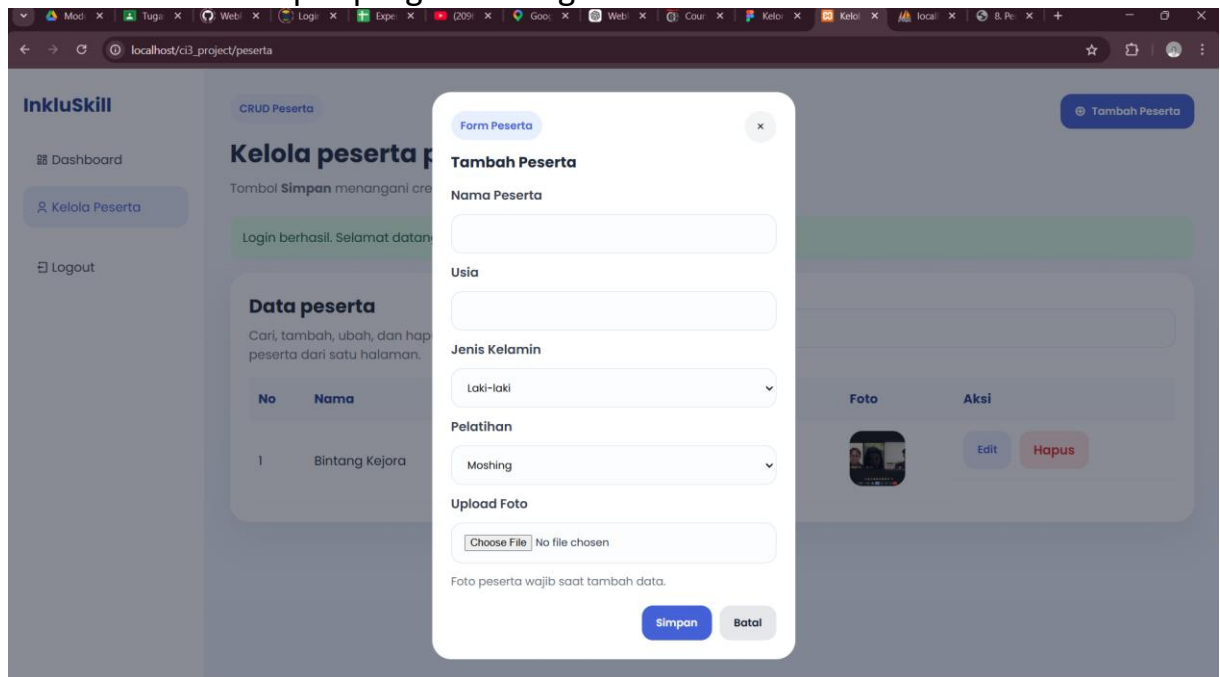
CREATE TABLE IF NOT EXISTS `ci3_participants` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `nama` varchar(150) NOT NULL,
  `umur` int(11) NOT NULL,
  `jenis_kelamin` varchar(20) NOT NULL,
  `pelatihan` varchar(100) NOT NULL,
  `foto` varchar(255) NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

INSERT INTO `ci3_users` (`username`, `email`, `password`, `role`)
SELECT 'Admin Inkluskill', 'admin@inkluskills.test',
'$2y$10$QRAHlJkQoY7M.nqFCyJ.50./tc6vAl/SNS9UK3hjenqKJ48L7Mqo.', 'sekolah'
WHERE NOT EXISTS (
  SELECT 1 FROM `ci3_users` WHERE `email` = 'admin@inkluskills.test'
);

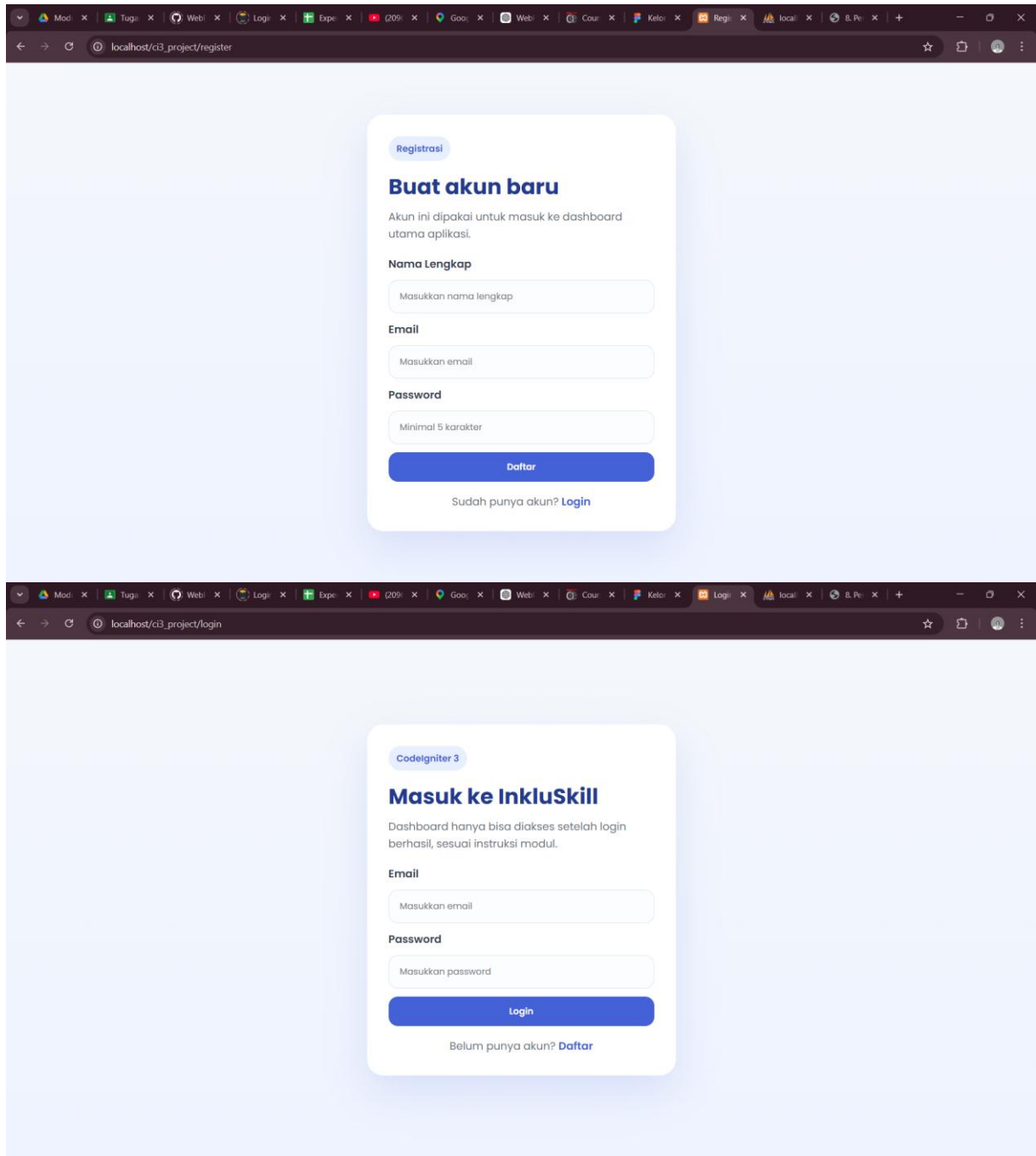
```



ini adalah hasil output program dibagian dashboard



Ini adalah program edit CRUD nya



Ini adalah Masuk dan Register nya

The screenshot shows the phpMyAdmin interface with the 'Table structure' tab selected for the 'c3_participants' table. The table has 8 columns: id, nama, umur, jenis_kelamin, pelatihan, foto, created_at, and updated_at. The 'id' column is the primary key. Below the table structure, the 'Indexes' section shows a primary index on the 'id' column.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	nama	varchar(150)	utf8mb4_general_ci		No	None			Change Drop More
3	umur	int(11)			No	None			Change Drop More
4	jenis_kelamin	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
5	pelatihan	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
6	foto	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
7	created_at	timestamp			No	current_timestamp()			Change Drop More
8	updated_at	timestamp			No	current_timestamp()		ON UPDATE CURRENT_TIMESTAMP()	Change Drop More

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY		BTREE	Yes	No	id	1	A	No	

Ini adalah struktur database participants nya

The screenshot shows the phpMyAdmin interface with the 'Browse' tab selected for the 'c3_participants' table. It displays a single row of data for a participant named 'Bintang Kejora'.

id	nama	umur	jenis_kelamin	pelatihan	foto	created_at	updated_at
1	Bintang Kejora	18	Laki-laki	Moshing	c8d280988d7834b75d50808198617452.jpeg	2026-04-22 21:29:14	2026-04-22 21:29:14

Ini adalah isi database participants nya

Table structure for database: webpro, table: c3_users

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	username	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
3	email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
4	password	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
5	role	varchar(50)	utf8mb4_general_ci		No	sekolah			Change Drop More
6	created_at	timestamp			No	current_timestamp()			Change Drop More

Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	2	A	No	
Edit Rename Drop	email	BTREE	Yes	No	email	2	A	No	

Ini adalah struktur database users nya

Showing rows 0 - 1 (2 total, Query took 0.0008 seconds)

SELECT * FROM `c3\_users`

Extra options

	id	username	email	password	role	created_at
☐ Edit ☐ Copy ☐ Delete	1	Admin InkuSkill	admin@inkuskill.test	\$2y\$10\$QRAHjKQoY7M.nqCyl.50.t6vAlSNS9UK3h9eq...	sekolah	2026-04-22 21:27:56
☐ Edit ☐ Copy ☐ Delete	2	Dryhuss	dzacky98ku@gmail.com	\$2y\$10\$dy7/oJPF7vScRDqWgxTr3.fYn4v.yMx6GKhDjw7Tzd5...	sekolah	2026-04-22 21:28:13

Ini adalah isi database users nya

E. Analisis dan Pembahasan

Berdasarkan hasil praktikum yang telah dilakukan, aplikasi web sederhana berbasis CodeIgniter 3 berhasil dibangun dengan menerapkan konsep MVC (Model View Controller). Penerapan konsep ini membuat struktur program menjadi lebih teratur karena bagian logika, tampilan, dan pengolahan data dipisahkan ke dalam file yang berbeda. Controller digunakan untuk mengatur alur proses, Model digunakan untuk mengelola data pada database, sedangkan View digunakan untuk menampilkan antarmuka kepada pengguna.

Aplikasi yang dibuat telah memenuhi kebutuhan utama praktikum, yaitu memiliki fitur login, dashboard, CRUD data peserta, dan upload file. Halaman dashboard hanya dapat diakses setelah proses login berhasil, sehingga sistem sudah menerapkan pembatasan akses menggunakan session. Selain itu, proses tambah dan edit data

peserta dapat dilakukan dengan satu tombol Simpan, termasuk untuk upload file foto dalam satu kali submit, sehingga sesuai dengan instruksi modul.

Dari sisi pengujian, fitur-fitur utama aplikasi dapat berjalan dengan baik, seperti registrasi akun, login, tambah data peserta, edit data, hapus data, dan upload foto. Validasi juga telah diterapkan pada input form dan file upload, misalnya pembatasan tipe file gambar dan ukuran maksimal file. Dengan demikian, aplikasi yang dibuat tidak hanya berfungsi secara teknis, tetapi juga sudah cukup baik dari sisi pengelolaan data dan kemudahan penggunaan.

Secara keseluruhan, praktikum ini memberikan pemahaman bahwa penggunaan framework CodeIgniter 3 dapat mempermudah proses pengembangan aplikasi web karena struktur kode lebih rapi, proses pengembangan lebih terarah, dan fitur-fitur seperti routing, session, validasi form, serta upload file sudah didukung oleh framework.

F. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa aplikasi web sederhana berbasis CodeIgniter 3 berhasil dibuat dengan menerapkan konsep MVC (Model View Controller). Aplikasi ini telah memenuhi kebutuhan utama praktikum, yaitu memiliki fitur login, dashboard, CRUD data peserta, serta upload file dalam satu kali proses submit menggunakan tombol Simpan.

Melalui praktikum ini dapat dipahami bahwa penggunaan framework CodeIgniter 3 membantu proses pengembangan aplikasi web menjadi lebih terstruktur, rapi, dan mudah dikelola. Selain itu, praktikum ini juga memberikan pemahaman tentang cara menghubungkan aplikasi dengan database, mengelola session, melakukan validasi form, serta menangani proses upload file pada aplikasi web.

Secara keseluruhan, tujuan praktikum telah tercapai dengan baik karena aplikasi yang dibuat dapat berjalan sesuai instruksi modul dan dapat digunakan sebagai dasar pengembangan tugas besar selanjutnya.

LINK REPOSITORY GITHUB DRYHUS

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/DryhusDzackyDamingtyas.S-707012530010>